

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CƠ SỞ TRÍ TUỆ NHÂN TẠO
MACHINE LEARNING: OVERFITTING PROBLEM

Giảng viên hướng dẫn: Thầy Lê Ngọc Thành
Thầy Huỳnh Lâm Hải Đăng
Thầy Lê Nhựt Nam

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Nguyễn Nhựt Huy	24127398	24C11
Trần Quang Minh Huy	24127403	24C11
Trịnh Khánh Linh	24127437	24C11
Nguyễn Minh Khôi	24127066	24C11

TP. Hồ Chí Minh, tháng 2/2026

THÔNG TIN VỀ ĐỒ ÁN

Mã học phần: CSC14003

Tên học phần: Cơ sở trí tuệ nhân tạo

Tên đồ án: Bài thu hoạch về vấn đề Overfitting

Hình thức: Bài tập nhóm

Giảng viên: thầy Lê Ngọc Thành, thầy Huỳnh Lâm Hải Đăng, thầy Lê Nhựt Nam

Sinh viên thực hiện:

Nguyễn Nhựt Huy 24127398

Trần Quang Minh Huy 24127403

Trịnh Khánh Linh 24127437

Nguyễn Minh Khôi 24127066

Mục lục

Danh sách hình vẽ	1
Danh sách bảng	2
1 Phân công công việc	3
1.1 Phân công chi tiết	3
2 Mở đầu	4
3 Bản chất và cách nhận diện Overfitting	5
3.1 Khái niệm và bản chất của Overfitting	5
3.2 Dấu hiệu nhận diện Overfitting	5
3.3 Vai trò của nhiễu trong Overfitting	5
3.4 Tổng hợp các dấu hiệu nhận biết	6
4 Các nguyên nhân chính gây ra Overfitting (Noise)	7
4.1 Stochastic Noise	7
4.1.1 Định nghĩa và bản chất của Stochastic Noise (Nhiều ngẫu nhiên)	7
4.1.2 Cơ chế Stochastic Noise dẫn đến Overfitting	7
4.1.3 Vai trò kép của Stochastic Noise trong quá trình huấn luyện	8
4.1.4 Minh chứng thực nghiệm	9
4.2 Deterministic Noise	9
4.2.1 Định nghĩa và bản chất của Deterministic Noise (Nhiều xác định)	9
4.2.2 Cơ chế Deterministic Noise dẫn đến Overfitting	10
4.2.3 Vai trò của Deterministic Noise trong quá trình huấn luyện	10
4.2.4 Minh chứng thực nghiệm	10
4.2.5 Cách giảm ảnh hưởng của deterministic noise	11
4.2.6 Nhận xét	11
4.2.7 Kết luận	11
5 Các yếu tố ảnh hưởng đến mức độ Overfitting	12
5.1 Nhóm các yếu tố về dữ liệu (data-driven factors)	12
5.1.1 Số lượng dữ liệu huấn luyện	12
5.1.2 Hiện tượng học nhiễu	13
5.1.3 Tỷ lệ đặc trưng trên mẫu (Feature-to-Sample Ratio – p/n)	13
5.2 Nhóm các yếu tố về mô hình (model-driven factors)	14
5.2.1 Sự nhạy cảm với sai lệch và độ phức tạp mô hình	14
5.2.2 Ngưỡng nội suy và góc nhìn hiện đại (Double Descent)	14
5.2.3 Phương pháp ước lượng tham số (Estimation Method)	15
5.3 Nhóm yếu tố về quy trình và thuật toán	15
5.3.1 Sự hiện diện của các điểm dị biệt (Outliers)	15

5.3.2	Sai lầm trong quy trình kiểm chứng	16
5.3.3	Thủ tục so sánh đa phương (Multiple Comparisons Procedures - MCP)	16
6	Các phương pháp khắc phục Overfitting	17
6.1	Regularization (Chỉnh hóa/Điều tiết)	17
6.1.1	L2 Regularization (Weight Decay/Ridge Regression)	17
6.1.2	L1 Regularization (Lasso)	18
6.2	Early Stopping (Dừng sớm)	19
6.3	Mở rộng tập dữ liệu (Data Augmentation)	20
6.4	Validation và Cross-Validation (Kiểm chứng chéo)	21
6.4.1	K-fold Cross-Validation	21
6.5	Lấy thêm dữ liệu (Data Collection)	22
	Tài liệu tham khảo	24

Danh sách hình vẽ

- 5.1 Mô hình so với đường dữ liệu thật ở kích thước dữ liệu nhỏ 12
- 5.2 Mô hình so với đường dữ liệu thật ở kích thước dữ liệu lớn 13
- 6.1 Dữ liệu huấn luyện và hàm sin gốc 18
- 6.2 Kết quả khớp đa thức bậc 9 với L2 regularization 18
- 6.3 Sai số huấn luyện và kiểm định theo số bước lặp (epoch) 19
- 6.4 Đường cong học tập trên MNIST: lỗi log-âm khả năng 20
- 6.5 Minh họa làm cong tổng hợp (synthetic warping) chữ số viết tay 21
- 6.6 Kỹ thuật kiểm định chéo S-fold ($S=4$) 22
- 6.7 Ảnh hưởng của kích thước tập dữ liệu lên overfitting (đa thức bậc 9) 23

Danh sách bảng

1.1 Phân công công việc các thành viên 3

Phần 1

Phân công công việc

1.1 Phân công chi tiết

Bảng 1.1: Phân công công việc các thành viên

Thành viên	Công việc được giao	Hoàn thành
Nguyễn Nhật Huy	Viết báo cáo phần Các yếu tố ảnh hưởng đến Overfitting và Kết luận	100%
Trần Quang Minh Huy	Viết báo cáo phần Các nguyên nhân gây ra Overfitting	100%
Trịnh Khánh Linh	Viết báo cáo phần Mở đầu và Bản chất, cách nhận diện Overfitting	100%
Nguyễn Minh Khôi	Viết báo phần Các phương pháp khắc phục Overfitting	100%

Phần 2

Mở đầu

Trong lĩnh vực Machine Learning, đặc biệt là học có giám sát (supervised learning), một trong những vấn đề quan trọng ảnh hưởng trực tiếp đến hiệu quả của mô hình là hiện tượng *overfitting*. Overfitting xảy ra khi mô hình học quá chi tiết từ dữ liệu huấn luyện, bao gồm cả các nhiễu (noise), dẫn đến khả năng tổng quát hóa kém trên dữ liệu chưa từng thấy. Nói cách khác, mô hình có thể đạt độ chính xác rất cao trên tập huấn luyện nhưng lại hoạt động kém trên tập kiểm tra [1].

Về bản chất, mục tiêu của Machine Learning không phải là tối ưu hóa tuyệt đối trên dữ liệu huấn luyện, mà là tối đa hóa khả năng dự đoán trên dữ liệu mới. Tuy nhiên, nếu mô hình được tối ưu hóa quá mức để giảm sai số trên tập huấn luyện, nó có xu hướng ghi nhớ các đặc điểm riêng lẻ hoặc nhiễu của dữ liệu thay vì học được quy luật tổng quát. Điều này dẫn đến sự sai lệch giữa mục tiêu tối ưu hóa và mục tiêu thực tế của bài toán học máy [2].

Tầm quan trọng của việc xử lý overfitting nằm ở khả năng quyết định tính ứng dụng thực tế của mô hình. Một mô hình chỉ hoạt động tốt trên dữ liệu huấn luyện nhưng thất bại trên dữ liệu thực tế sẽ không có giá trị sử dụng. Do đó, việc nhận diện và kiểm soát overfitting là một kỹ năng cốt lõi trong quá trình xây dựng mô hình học máy.

Một dấu hiệu phổ biến của overfitting là khi sai số trên tập huấn luyện (E_{in}) tiếp tục giảm, trong khi sai số trên tập kiểm tra (E_{out}) lại bắt đầu tăng. Hiện tượng này cho thấy mô hình đang dần học cả những yếu tố không mang tính quy luật của dữ liệu.

Phần 3

Bản chất và cách nhận diện Overfitting

3.1 Khái niệm và bản chất của Overfitting

Overfitting là một khái niệm mang tính tương đối và so sánh. Một mô hình được coi là bị overfit khi nó có hiệu suất tốt hơn trên tập huấn luyện nhưng lại kém hơn trên dữ liệu ngoài mẫu so với một mô hình khác, hoặc so với chính nó tại một thời điểm trước đó trong quá trình huấn luyện.

Về bản chất, overfitting phản ánh sự mất cân bằng giữa việc học dữ liệu huấn luyện và khả năng tổng quát hóa. Mục tiêu của Machine Learning không phải là tối ưu hóa tuyệt đối trên tập huấn luyện mà là đạt hiệu suất tốt trên dữ liệu chưa từng thấy. Khi mô hình quá tập trung vào việc giảm sai số huấn luyện, nó có xu hướng ghi nhớ các đặc điểm riêng lẻ hoặc nhiễu trong dữ liệu thay vì học được quy luật tổng quát.

Cần phân biệt rõ giữa overfitting và khả năng tổng quát hóa kém. Một mô hình có thể có sai số cao trên cả E_{in} và E_{out} , nhưng trường hợp này không được xem là overfitting. Overfitting chỉ xảy ra khi mô hình tối ưu hóa quá mức E_{in} dẫn đến làm giảm hiệu suất trên E_{out} .

3.2 Dấu hiệu nhận diện Overfitting

Một trong những dấu hiệu quan trọng nhất để nhận diện overfitting là mối quan hệ giữa sai số huấn luyện và sai số kiểm tra. Theo Abu-Mostafa trong *Learning From Data* (Lecture 11: *Overfitting*), khi sai số trong mẫu E_{in} tiếp tục giảm nhưng sai số ngoài mẫu E_{out} lại tăng (" $E_{in} \downarrow E_{out} \uparrow$ "), đó là dấu hiệu mô hình đang rơi vào hiện tượng overfitting, tức học cả nhiễu trong dữ liệu huấn luyện thay vì quy luật tổng quát [3]. Nói cách khác, mô hình không chỉ học quy luật thực sự mà còn "fit" cả những biến động ngẫu nhiên không có ý nghĩa.

Hiện tượng này thường xuất hiện rõ ràng trong quá trình huấn luyện các mô hình học sâu. Ban đầu, khi mô hình bắt đầu học, cả E_{in} và E_{out} đều giảm do mô hình dần nắm bắt được các đặc trưng quan trọng của dữ liệu. Tuy nhiên, sau một số epoch nhất định, việc tiếp tục huấn luyện sẽ khiến mô hình bắt đầu ghi nhớ dữ liệu huấn luyện, dẫn đến việc E_{out} tăng lên trong khi E_{in} vẫn tiếp tục giảm. Đây chính là thời điểm mà overfitting bắt đầu xảy ra.

Ngoài ra, một số biểu hiện khác của overfitting có thể bao gồm việc mô hình trở nên quá phức tạp so với lượng dữ liệu hiện có, hoặc hiệu suất trên dữ liệu mới kém dù kết quả trên dữ liệu huấn luyện rất tốt.

3.3 Vai trò của nhiễu trong Overfitting

Một nguyên nhân cốt lõi của overfitting là việc mô hình học cả nhiễu trong dữ liệu. Theo [1], các mô hình bị overfit có xu hướng ghi nhớ toàn bộ dữ liệu huấn luyện, bao gồm cả những thành phần không mang tính quy luật. Điều này khiến mô hình không thể tổng quát hóa tốt khi gặp dữ liệu mới.

Nhiều trong dữ liệu có thể đến từ nhiều nguồn khác nhau, chẳng hạn như sai số đo lường, dữ liệu không đầy đủ hoặc các biến ngẫu nhiên không thể dự đoán. Khi mô hình cố gắng khớp với các dữ liệu này, nó sẽ học những mẫu không có ý nghĩa thực tế, từ đó làm giảm hiệu suất tổng thể.

Do đó, việc kiểm soát mức độ phức tạp của mô hình và hiểu rõ vai trò của nhiễu là yếu tố quan trọng trong việc hạn chế overfitting.

3.4 Tổng hợp các dấu hiệu nhận biết

Từ các phân tích trên, có thể tổng hợp một số dấu hiệu chính để nhận diện overfitting như sau:

- Sai số huấn luyện giảm nhưng sai số kiểm tra tăng.
- Mô hình quá phức tạp so với dữ liệu.
- Mô hình học cả nhiễu thay vì quy luật.
- Hiệu suất trên dữ liệu mới kém dù kết quả trên dữ liệu huấn luyện rất tốt.

Phần 4

Các nguyên nhân chính gây ra Overfitting (Noise)

4.1 Stochastic Noise

4.1.1 Định nghĩa và bản chất của Stochastic Noise (Nhiều ngẫu nhiên)

Khái niệm Trong học máy, các điểm dữ liệu huấn luyện thường không phản ánh chính xác hàm mục tiêu thực f^* mà chỉ là các ví dụ xấp xỉ có chứa nhiễu (*noisy approximate examples*) [4]. Mỗi điểm huấn luyện bao gồm một giá trị đầu vào x nằm trong miền xác định và một nhãn tương ứng $y^* \approx f^*(x)$.

Trong [5] đã đề cập về các mô hình hồi quy, hiện tượng này thường được biểu diễn dưới dạng:

$$y = g(x) + \eta$$

trong đó $g(x)$ là hàm thực cần tìm và η là thành phần nhiễu ngẫu nhiên với kỳ vọng bằng không:

$$\mathbb{E}[\eta] = 0$$

Thành phần nhiễu [6] Nhiễu được định nghĩa là thành phần không thể tránh khỏi của sai số (*unavoidable component of loss*). Đặc điểm quan trọng nhất của thành phần này là nó tồn tại độc lập với thuật toán học (*incurred independently of the learning algorithm*).

Nói cách khác, ngay cả khi sai số do bias và variance được giảm thiểu hoàn toàn, thành phần nhiễu vẫn luôn tồn tại như một giới hạn nội tại của dữ liệu.

Nguồn gốc của nhiễu ngẫu nhiên Nhiễu ngẫu nhiên trong dữ liệu phát sinh từ nhiều yếu tố khách quan.

- **Sai số đo lường và thực nghiệm:** Các dữ liệu thu thập từ đo đạc thực tế hoặc các phép tính toán mô phỏng thường bị giới hạn bởi độ chính xác của thiết bị hoặc các ràng buộc về mặt công cụ.
- **Biến số không thể kiểm soát:** Nhiễu có thể đến từ sự biến thiên của các yếu tố ngẫu nhiên trong quá trình thu thập dữ liệu thực địa, khiến cho cùng một đầu vào x có thể dẫn đến các giá trị đầu ra t khác nhau, tức là hàm ánh xạ thực tế không còn là hàm đơn trị tuyệt đối.
- **Đặc thù của dữ liệu thô:** Các dữ liệu trong khoa học và kỹ thuật thường ở trạng thái “thô”, chứa đựng các dao động ngẫu nhiên (*fluctuations*) không mang thông tin về quy luật tổng quát của hệ thống.

4.1.2 Cơ chế Stochastic Noise dẫn đến Overfitting

Học thuộc lòng nhiễu (Learning the Idiosyncrasies) Khi một mô hình có khả năng biểu diễn (*capacity*) lớn được huấn luyện, nó không chỉ cố gắng tìm ra hàm mục tiêu thực f^* mà còn bám theo các biến động ngẫu nhiên trong tập dữ liệu cụ thể đó.

Hiện tượng overfitting xảy ra khi sai số huấn luyện tiếp tục giảm nhưng sai số kiểm thử bắt đầu tăng lên, do mô hình đã bắt đầu “học thuộc lòng các đặc điểm riêng biệt” (*learning idiosyncrasies*) [7] của dữ liệu huấn luyện thay vì các quy luật tổng quát.

Hệ quả của sự dư thừa tham số (*Over-parameterization*) Các mạng nơ-ron sâu với số lượng tham số rất lớn, chẳng hạn như các mô hình LDNN (*Large Deep Neural Networks*), có khả năng biểu diễn rất cao, cho phép chúng nội suy hoàn toàn dữ liệu huấn luyện, bao gồm cả nhiễu.

Sự dư thừa tham số khiến mô hình trở nên cực kỳ nhạy cảm với các điểm dữ liệu cụ thể; nếu mô hình quá phức tạp so với lượng dữ liệu hiện có, nó sẽ “nội suy dữ liệu” (*interpolate the data*) [5] một cách quá mức, dẫn đến nguy cơ đóng góp sai số phương sai (*variance*) rất lớn vào tổng sai số của hệ thống.

Góc nhìn Bias–Variance Trong lý thuyết phân rã sai số (*Bias–Variance Decomposition*) [6], tổng sai số dự đoán thường được chia thành ba thành phần chính: Bias, Variance và Noise.

Bias đại diện cho sai số hệ thống do giả định mô hình quá đơn giản, trong khi Variance (phương sai) đo lường mức độ nhạy cảm của mô hình đối với các biến động trong tập dữ liệu huấn luyện.

Stochastic noise trong dữ liệu đầu vào làm tăng Variance, bởi vì các mô hình phức tạp sẽ thay đổi kết quả dự đoán một cách mạnh mẽ khi được huấn luyện trên các tập dữ liệu nhiễu khác nhau.

4.1.3 Vai trò kép của Stochastic Noise trong quá trình huấn luyện

Nhiều trong dữ liệu và nhiễu trong thuật toán Cần phân biệt rõ hai loại nhiễu trong học máy.

Nhiều dữ liệu (*stochastic noise in data*) là thành phần sai số không thể tránh khỏi, phát sinh độc lập với thuật toán học. Nó thường đến từ sai số đo lường, giới hạn của thiết bị hoặc các biến số không thể kiểm soát trong quá trình thu thập dữ liệu.

Ngược lại, **nhiều thuật toán**, điển hình là *gradient noise* trong SGD, xuất hiện do việc lấy mẫu ngẫu nhiên các mini-batch để xấp xỉ gradient của toàn bộ tập huấn luyện [7].

Trong khi nhiễu dữ liệu là nguyên nhân chính gây ra overfitting, nhiễu gradient lại có vai trò tích cực trong quá trình tối ưu hóa.

Tác dụng của Gradient Noise như một dạng Implicit Regularization Khác với nhiễu dữ liệu làm mô hình học sai lệch, nhiễu gradient hoạt động như một cơ chế điều chuẩn ẩn (*implicit regularization*) [8].

- **Thoát khỏi cực tiểu nhọn (*Sharp Minima*):** Nhiễu gradient giúp mô hình tránh bị mắc kẹt tại các cực tiểu nhọn, nơi hàm mất mát thay đổi mạnh chỉ với một biến động nhỏ của tham số. Những vùng này thường dẫn đến khả năng tổng quát hóa kém.
- **Hướng tới cực tiểu phẳng (*Flat Minima*):** Sự ngẫu nhiên trong quá trình tối ưu giúp mô hình ưu tiên hội tụ về các cực tiểu phẳng, nơi các giá trị riêng của ma trận Hessian nhỏ hơn. Khi đó mô hình ít nhạy cảm hơn với biến động nhỏ của dữ liệu và có khả năng tổng quát hóa tốt hơn.

Sự cân bằng giữa nhiễu và độ chính xác Mặc dù nhiễu dữ liệu làm tăng nguy cơ học thuộc lòng các đặc điểm ngẫu nhiên của tập huấn luyện, việc duy trì một mức độ nhiễu phù hợp trong tối ưu hóa lại là cần thiết.

Thực nghiệm cho thấy *small-batch training* tạo ra đủ nhiễu gradient để mô hình tránh bám quá chặt vào dữ liệu nhiễu. Ngược lại, *large-batch training* làm giảm nhiễu gradient, khiến mô hình dễ rơi vào các cực tiểu nhọn và làm tăng *generalization gap* [7].

Vì vậy, việc cân bằng giữa tốc độ học (*learning rate*) và kích thước batch là yếu tố quan trọng để tận dụng lợi ích của stochastic noise trong quá trình huấn luyện.

4.1.4 Minh chứng thực nghiệm

Mô hình LDNN và SDNN: Dư thừa tham số và overfitting nhiều [4] Các thực nghiệm trên những hàm mục tiêu có chứa nhiễu cho thấy sự khác biệt rõ rệt giữa mạng nơ-ron nhỏ (SDNN – *Small Deep Neural Network*) và mạng nơ-ron lớn (LDNN – *Large Deep Neural Network*).

SDNN với số lượng tham số nhỏ thường đạt hiệu suất tốt hơn, thể hiện qua MSE thấp hơn và SNR cao hơn trong hầu hết các kịch bản có nhiễu. Ngược lại, LDNN với số lượng tham số rất lớn dễ bị hiện tượng dư thừa tham số (*over-parameterization*), khiến mô hình có xu hướng học thuộc cả các thành phần nhiễu thay vì chỉ học hàm mục tiêu thực.

Kích thước batch và Generalization Gap [7] Các nghiên cứu thực nghiệm giữa huấn luyện với batch nhỏ (*small-batch*) và batch lớn (*large-batch*) cho thấy sự tồn tại rõ rệt của *generalization gap*.

Với *small-batch training*, nhiễu gradient lớn hơn giúp mô hình thoát khỏi các cực tiểu nhọn (*sharp minima*), tức những vùng mà hàm mất mát thay đổi mạnh khi tham số chỉ biến động nhỏ. Các cực tiểu này thường dẫn đến khả năng tổng quát hóa kém.

Ngược lại, khi sử dụng *large-batch training*, nhiễu gradient giảm đáng kể, khiến thuật toán dễ bị mắc kẹt tại các cực tiểu nhọn gần điểm khởi tạo thay vì hội tụ tới các cực tiểu phẳng (*flat minima*) có khả năng tổng quát hóa tốt hơn.

Sự tồn tại của ngưỡng batch size [7] Thực nghiệm cũng cho thấy tồn tại một ngưỡng kích thước batch, sau đó độ chính xác trên tập kiểm thử giảm mạnh trong khi độ nhọn (*sharpness*) của cực tiểu tăng lên đáng kể.

Điều này chứng minh rằng việc giảm quá mức stochastic noise trong quá trình tối ưu có thể khiến mô hình bám quá chặt vào dữ liệu huấn luyện và làm suy giảm hiệu suất trên dữ liệu mới.

4.2 Deterministic Noise

4.2.1 Định nghĩa và bản chất của Deterministic Noise (Nhiều xác định)

Khái niệm Trong nhiều bài toán học máy, sai số dự đoán không chỉ đến từ các yếu tố ngẫu nhiên mà còn xuất phát từ giới hạn biểu diễn của mô hình. Khi tập giả thuyết H không đủ linh hoạt để mô tả chính xác hàm mục tiêu thực $f(x)$, phần sai lệch còn lại có thể được xem là *deterministic noise*, hay nhiễu xác định [5].

Dưới góc nhìn xấp xỉ hàm, *deterministic noise* thường được mô tả bởi:

$$\text{Deterministic Noise} = f(x) - h^*(x) \tag{4.1}$$

trong đó:

- $f(x)$: hàm mục tiêu thực sự của dữ liệu.
- $h^*(x)$: giả thuyết tốt nhất có thể đạt được trong tập giả thuyết H .

Bản chất của Deterministic Noise Bản chất của sai số này là có cấu trúc, lặp lại được và phụ thuộc trực tiếp vào lựa chọn mô hình. Nếu mở rộng không gian giả thuyết hoặc bổ sung đặc trưng phù hợp, *deterministic noise* có thể giảm đáng kể.

Khác với *stochastic noise* vốn tồn tại độc lập với thuật toán học, *deterministic noise* phản ánh sai số hệ thống do mô hình chưa đủ năng lực biểu diễn quy luật thực tế [6].

Nguồn gốc của Deterministic Noise Nguồn gốc của deterministic noise thường đến từ các nguyên nhân sau:

- **Mô hình quá đơn giản:** Ví dụ dùng hồi quy tuyến tính cho dữ liệu phi tuyến.
- **Thiếu đặc trưng quan trọng:** Các biến đầu vào chưa phản ánh đầy đủ bản chất dữ liệu.
- **Giả định sai về cấu trúc dữ liệu:** Chọn mô hình không phù hợp với phân phối thực tế.
- **Ràng buộc quá mạnh:** Regularization quá mức làm giảm khả năng học tín hiệu thật.

4.2.2 Cơ chế Deterministic Noise dẫn đến Overfitting

Học sai lệch cấu trúc Khi deterministic noise tồn tại, mô hình không thể học hoàn chỉnh hàm mục tiêu thật mà chỉ quan sát được phần dữ liệu đã bị sai lệch bởi giới hạn biểu diễn. Nếu tiếp tục tăng độ phức tạp mô hình để giảm sai số huấn luyện, thuật toán có xu hướng khớp cả những sai lệch mang tính cục bộ này thay vì quy luật tổng quát.

Hiện tượng overfitting khi đó xuất hiện khi sai số huấn luyện tiếp tục giảm nhưng sai số kiểm thử không cải thiện hoặc bắt đầu tăng lên. Mô hình đã học tốt tập huấn luyện, nhưng học trên một biểu diễn chưa chính xác của dữ liệu [2, p. 326].

Hệ quả của việc tăng độ phức tạp mô hình Nếu mô hình được mở rộng quá mức nhằm bù đắp deterministic noise, số bậc tự do tăng lên sẽ làm variance lớn hơn. Điều này khiến mô hình trở nên nhạy cảm với dao động nhỏ của dữ liệu huấn luyện và suy giảm khả năng tổng quát hóa [5].

Góc nhìn Bias–Variance Deterministic noise thường gắn với bias cao ở giai đoạn đầu, khi mô hình quá đơn giản. Tuy nhiên, nếu cố giảm bias bằng cách tăng complexity quá mức, mô hình có thể chuyển sang trạng thái variance cao và dẫn đến overfitting.

4.2.3 Vai trò của Deterministic Noise trong quá trình huấn luyện

Giới hạn của việc tối ưu hóa Một thuật toán tối ưu tốt không đồng nghĩa với mô hình tốt. Ngay cả khi quá trình huấn luyện tìm được nghiệm tối ưu trên tập dữ liệu, deterministic noise vẫn tồn tại nếu cấu trúc mô hình chưa phù hợp.

Tín hiệu cho việc lựa chọn mô hình Sự tồn tại của deterministic noise cho thấy cần quan tâm đến model selection thay vì chỉ tối ưu tham số. Trong nhiều trường hợp, thay đổi kiến trúc mô hình hoặc bổ sung đặc trưng hiệu quả hơn việc tiếp tục giảm loss.

Mối liên hệ với regularization Nếu regularization quá mạnh, deterministic noise có thể tăng do mô hình bị hạn chế quá mức. Ngược lại, regularization hợp lý giúp cân bằng giữa bias và variance, từ đó giảm nguy cơ overfitting [1].

4.2.4 Minh chứng thực nghiệm

Ví dụ khớp hàm phi tuyến Giả sử dữ liệu được sinh từ hàm:

$$y = \sin(\pi x)$$

Nếu sử dụng hồi quy tuyến tính để xấp xỉ, mô hình sẽ không thể nắm bắt dạng cong của hàm mục tiêu. Phần sai lệch giữa đường thẳng dự đoán và đường cong thực chính là deterministic noise.

So sánh mô hình đơn giản và phức tạp Khi thay mô hình tuyến tính bằng đa thức bậc cao hơn, deterministic noise giảm do năng lực biểu diễn tăng. Tuy nhiên, nếu bậc đa thức quá lớn so với số lượng mẫu, mô hình lại dễ học cả dao động ngẫu nhiên trong dữ liệu và dẫn đến overfitting [9].

Kết luận thực nghiệm Kết quả thực nghiệm này cho thấy deterministic noise và stochastic noise thường cùng tồn tại. Việc giảm một loại sai số bằng cách tăng độ phức tạp mô hình có thể làm gia tăng loại sai số còn lại nếu không được kiểm soát hợp lý.

4.2.5 Cách giảm ảnh hưởng của deterministic noise

- Lựa chọn tập giả thuyết phù hợp hơn với dữ liệu.
- Thiết kế đặc trưng tốt hơn để mô hình đơn giản vẫn học được quy luật.
- Sử dụng validation để chọn độ phức tạp phù hợp.
- Áp dụng regularization nhằm kiểm soát khả năng khớp quá mức [1].

4.2.6 Nhận xét

- Không phải mọi sai số đều đến từ dữ liệu nhiễu; nhiều trường hợp xuất phát từ việc chọn mô hình chưa phù hợp.
- Trong thực tế, deterministic noise gần như luôn tồn tại vì hàm mục tiêu thật thường không quan sát trực tiếp được.
- Mô hình phức tạp hơn chưa chắc tốt hơn nếu dữ liệu không đủ lớn.

4.2.7 Kết luận

Deterministic noise là dạng sai số phát sinh do giới hạn biểu diễn của mô hình. Khi mô hình cố gắng giảm sai số huấn luyện bằng cách khớp cả phần sai lệch này, khả năng tổng quát hóa có thể suy giảm và dẫn đến overfitting. Vì vậy, việc cân bằng giữa độ phức tạp mô hình, dữ liệu huấn luyện và các kỹ thuật kiểm soát như validation hay regularization giữ vai trò quan trọng trong xây dựng hệ thống học máy hiệu quả.

Phần 5

Các yếu tố ảnh hưởng đến mức độ Overfitting

5.1 Nhóm các yếu tố về dữ liệu (data-driven factors)

5.1.1 Số lượng dữ liệu huấn luyện

5.1.1.1 Ảnh hưởng của kích thước mẫu nhỏ

Sự ghi nhớ thay vì học quy luật Khi kích thước tập huấn luyện nhỏ, mô hình thường đạt độ chính xác rất cao trên dữ liệu huấn luyện nhưng suy giảm mạnh trên tập kiểm tra. Nguyên nhân là do mô hình có xu hướng ghi nhớ cả nhiễu trong dữ liệu thay vì học các quy luật tổng quát, dẫn đến overfitting [1].

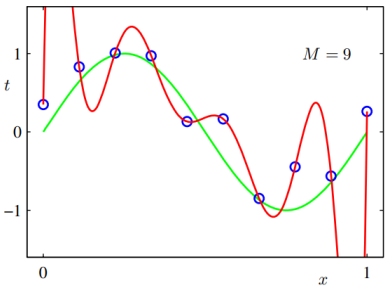
Hệ quả Trong các lĩnh vực như neuroimaging hay genomics, nơi số đặc trưng lớn nhưng số mẫu hạn chế, các nghiên cứu thường báo cáo độ chính xác cao bất thường [10]. Thực nghiệm cho thấy tồn tại mối tương quan nghịch giữa kích thước mẫu và độ chính xác, ví dụ với hệ số $r = -0.70$, cho thấy phần lớn biến thiên về độ chính xác có thể bị chi phối bởi quy mô dữ liệu.

Điều này phản ánh hiện tượng sai số lạc quan (optimistic bias), khi mô hình khớp với nhiễu trong dữ liệu nhỏ thay vì học được cấu trúc thực sự.

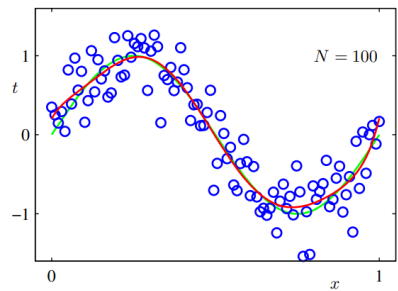
5.1.1.2 Mối liên hệ giữa N và độ phức tạp

Trong cuốn sách *Pattern Recognition and Machine Learning*, [9, pp. 4–9] đã sử dụng ví dụ về khớp đường cong đa thức (Polynomial Curve Fitting) để minh họa mối liên hệ giữa số lượng dữ liệu N và độ phức tạp của mô hình M .

Ảnh hưởng của N đến overfitting Như thể hiện trong Hình 5.1, khi số lượng dữ liệu nhỏ (ví dụ $N = 10$) và sử dụng đa thức bậc cao ($M = 9$), mô hình có thể khớp hoàn toàn dữ liệu huấn luyện nhưng dao động mạnh giữa các điểm, dẫn đến khả năng tổng quát hóa kém. Ngược lại, trong Hình 5.2, khi tăng số lượng dữ liệu (ví dụ $N = 100$), với cùng một giá trị M , hiện tượng overfitting giảm đáng kể do các ràng buộc từ dữ liệu trở nên mạnh hơn, hạn chế khả năng mô hình khớp với nhiễu.



Hình 5.1: Mô hình so với đường dữ liệu thật ở kích thước dữ liệu nhỏ



Hình 5.2: Mô hình so với đường dữ liệu thật ở kích thước dữ liệu lớn

5.1.2 Hiện tượng học nhiễu

Trong môi trường lý tưởng, các mô hình học máy được kỳ vọng sẽ tìm ra các quy luật tổng quát (disciplines) ẩn sau dữ liệu huấn luyện. Tuy nhiên, trên thực tế, dữ liệu luôn bị bao phủ bởi một lớp “nhiều” phát sinh từ các quá trình ngẫu nhiên hoặc các nguồn biến thiên không thể quan sát được.

5.1.2.1 Bản chất của hiện tượng học nhiễu (Noise Learning)

Học nhiễu xảy ra khi mô hình học cả những biến động ngẫu nhiên trong dữ liệu thay vì chỉ nắm bắt cấu trúc tổng quát[1].

- **Nhầm lẫn giữa tín hiệu và nhiễu.** Mô hình không phân biệt được đâu là quy luật thực và đâu là sai số ngẫu nhiên, đặc biệt khi dữ liệu chứa nhiều biến động nhỏ.
- **Hệ quả lên khả năng tổng quát hóa.** Việc khớp với nhiễu làm mô hình trở nên thiếu ổn định, dẫn đến sai số kiểm tra cao dù sai số huấn luyện thấp. [11, p. 221]

5.1.2.2 Các yếu tố thực nghiệm làm trầm trọng hiện tượng học nhiễu

- **Sai lầm trong quy trình kiểm chứng.** Việc chọn đặc trưng hoặc tối ưu tham số trên toàn bộ dữ liệu trước khi chia tập kiểm tra gây rò rỉ thông tin và làm sai lệch đánh giá[11, p. 245].
- **Quá trình tối ưu quá mức.** Huấn luyện quá lâu hoặc không có điều chuẩn (regularization) khiến mô hình dần khớp với nhiễu trong dữ liệu.

5.1.3 Tỷ lệ đặc trưng trên mẫu (Feature-to-Sample Ratio – p/n)

Tỷ lệ p/n phản ánh mối quan hệ giữa độ phức tạp biểu diễn và lượng thông tin quan sát, và có thể được xem là hệ quả trực tiếp của lời nguyền chiều dữ liệu.

Lời nguyền chiều dữ liệu (Curse of Dimensionality) Khi số lượng đặc trưng tăng lên, không gian đầu vào mở rộng theo hàm mũ, dẫn đến hiện tượng dữ liệu trở nên thưa thớt [9, pp. 33-38].

- **Sự bùng nổ của không gian.** Nếu mỗi chiều được chia thành M phần, thì trong không gian D chiều sẽ có M^D vùng. Do đó, số lượng dữ liệu cần thiết để bao phủ không gian này tăng theo hàm mũ, gây khó khăn cho việc ước lượng đáng tin cậy.
- **Mật độ lấy mẫu giảm mạnh.** Mật độ lấy mẫu hiệu quả giảm nhanh khi số chiều tăng, khiến khoảng cách trung bình giữa các điểm dữ liệu tăng lên đáng kể. Để đạt mật độ tương đương với 100 điểm trong

không gian một chiều, không gian 10 chiều sẽ cần tới 100^{10} điểm, vượt xa khả năng thu thập dữ liệu thực tế [11, p. 23].

Sự thưa thớt và vấn đề ở biên không gian Trong không gian nhiều chiều, phần lớn các điểm dữ liệu có xu hướng nằm gần biên của không gian mẫu [11, p. 23].

- **Lân cận mất tính cục bộ.** Để thu thập một tỷ lệ nhỏ dữ liệu (ví dụ 10%) trong không gian 10 chiều, cần bao phủ tới khoảng 80% phạm vi của mỗi biến. Khi đó, khái niệm “lân cận” không còn mang tính cục bộ, làm giảm hiệu quả của các phương pháp dựa trên khoảng cách [11, p. 22].
- **Ngoại suy thay vì nội suy.** Do dữ liệu tập trung ở biên, việc dự đoán cho điểm mới thường yêu cầu mô hình thực hiện ngoại suy (extrapolation), vốn tiềm ẩn rủi ro cao và dễ dẫn đến sai số lớn, đặc biệt khi mô hình đã khớp với nhiễu[11, p. 23]..

Nguy cơ xuất hiện các “quy luật giả” Tỷ lệ p/n cao cung cấp cho mô hình nhiều bậc tự do, làm tăng khả năng xuất hiện các tương quan ngẫu nhiên.

- **Khớp với tổ hợp nhiễu.** Khi số lượng đặc trưng vượt xa số lượng mẫu, mô hình dễ tìm ra các tổ hợp đặc trưng có vẻ giải thích tốt dữ liệu, nhưng thực chất chỉ là khớp với nhiễu [10, pp. 1, 3].
- **Sai lệch từ quy trình chọn lọc.** Nếu việc lựa chọn đặc trưng được thực hiện trên toàn bộ dữ liệu trước khi tách tập kiểm chứng, mô hình có thể tận dụng các tương quan ngẫu nhiên để đạt độ chính xác cao giả tạo, nhưng sẽ thất bại trên dữ liệu thực [10, p. 2, 12].
- **Ảnh hưởng của quy mô mẫu.** Trong các tập dữ liệu nhỏ với số đặc trưng lớn, một phần đáng kể biến thiên của độ chính xác có thể bị chi phối bởi chính kích thước mẫu thay vì chất lượng mô hình[10, p. 7].

5.2 Nhóm các yếu tố về mô hình (model-driven factors)

5.2.1 Sự nhạy cảm với sai lệch và độ phức tạp mô hình

Mức độ overfitting phụ thuộc chặt chẽ vào khả năng của mô hình trong việc phân biệt giữa dữ liệu đại diện và nhiễu.

- **Tác động của độ linh hoạt.** Các mô hình có độ phức tạp cao (ví dụ: đa thức bậc cao hoặc mạng thần kinh lớn) có nhiều bậc tự do, khiến chúng trở nên cực kỳ nhạy với các dao động nhỏ trong dữ liệu. Thay vì học một hàm trơn biểu diễn quy luật thực, mô hình có thể dao động mạnh để khớp chính xác các điểm nhiễu[9, pp. 4–12].
- **Sự đánh đổi Bias–Variance.** Dưới góc nhìn thống kê, sự nhạy cảm này làm tăng thành phần phương sai (variance) của mô hình. Một mô hình có phương sai cao sẽ thay đổi đáng kể dự đoán khi dữ liệu huấn luyện thay đổi nhẹ, phản ánh sự thiếu ổn định trước nhiễu.[12]

5.2.2 Ngưỡng nội suy và góc nhìn hiện đại (Double Descent)

Theo quan điểm cổ điển, mức độ overfitting tăng dần khi tỷ lệ p/n tăng. Tuy nhiên, các nghiên cứu gần đây đã phát hiện một hiện tượng mới được gọi là *double descent* [13].

- **Đỉnh overfitting tại ngưỡng nội suy.** Hiện tượng quá khớp đạt mức nghiêm trọng nhất tại ngưỡng nội suy, khi số lượng tham số xấp xỉ số lượng mẫu ($p \approx n$).

- **Cải thiện trong chế độ overparameterized.** Khi số lượng tham số vượt xa số lượng mẫu ($p \gg n$), sai số kiểm tra có thể giảm trở lại. Điều này được giải thích bởi việc các thuật toán tối ưu (như SGD) có xu hướng tìm được nghiệm có chuẩn nhỏ và trơn hơn, từ đó cải thiện khả năng khái quát hóa [14].

5.2.3 Phương pháp ước lượng tham số (Estimation Method)

Phương pháp Maximum Likelihood Estimation (MLE) tìm kiếm một ước lượng điểm (point estimate) cho các tham số sao cho xác suất thu được tập dữ liệu quan sát là lớn nhất. Tuy nhiên, cách tiếp cận này đối mặt với các vấn đề sau:

- **Tối ưu hóa cục bộ trên dữ liệu huấn luyện:** MLE giả định rằng các giá trị tham số làm cho dữ liệu huấn luyện có xác suất cao nhất chính là các giá trị thực tế. Điều này khiến mô hình cực kỳ nhạy cảm với nhiễu ngẫu nhiên trong tập dữ liệu nhỏ.
- **Sai lệch hệ thống (Bias) trong ước lượng:** Một ví dụ điển hình là việc ước lượng phương sai của phân phối Gaussian. MLE có xu hướng đánh giá thấp (underestimate) phương sai thực tế vì nó đo lường sự biến thiên dựa trên trung bình mẫu (vốn đã được điều chỉnh theo nhiễu của dữ liệu huấn luyện) thay vì trung bình thực của quần thể [9, pp. 26-28].
- **Vấn đề với dữ liệu phân tách hoàn hảo:** Trong các mô hình phân loại như Logistic Regression, nếu dữ liệu có thể phân tách tuyến tính một cách hoàn hảo, MLE sẽ đẩy các tham số tiến tới vô cùng để ép xác suất dự báo về 1 hoặc 0, dẫn đến mô hình bị quá khớp nghiêm trọng [9, pp. 206-207].

5.3 Nhóm yếu tố về quy trình và thuật toán

5.3.1 Sự hiện diện của các điểm dị biệt (Outliers)

Các điểm dị biệt là những quan sát không tuân theo phân phối chung của dữ liệu hoặc nằm xa ranh giới quyết định lý tưởng. Sự hiện diện của chúng trong tập huấn luyện là một trong những tác nhân quan trọng dẫn đến overfitting, đặc biệt đối với các mô hình dựa trên các giả định thống kê cứng nhắc.

Cơ chế kéo lệch mô hình (Model Pulling) Hiện tượng mô hình bị “kéo lệch” xuất phát từ bản chất của các hàm mất mát phổ biến

- **Sự nhạy cảm của bình phương tối thiểu (Least Squares).** Phương pháp bình phương tối thiểu thiếu tính bền vững (robustness) trước các điểm dị biệt. Do hàm mất mát sử dụng bình phương sai số, các quan sát có sai số lớn sẽ có ảnh hưởng rất mạnh đến quá trình tối ưu[9, p. 185].
- **Mối liên hệ với phân phối Gaussian.** Việc sử dụng bình phương tối thiểu tương đương với ước lượng hợp lý cực đại (Maximum Likelihood) dưới giả định nhiễu tuân theo phân phối Gaussian. Do phân phối này có “đuôi nhẹ”, nó không mô hình hóa tốt các điểm nằm xa trung tâm, khiến tham số ước lượng bị lệch đáng kể khi xuất hiện outliers[9, p. 29, 103].

Tác động đến ranh giới quyết định Để giảm sai số huấn luyện, mô hình sẽ cố gắng điều chỉnh tham số nhằm khớp với cả các điểm dị biệt:

- **Điều chỉnh cực đoan.** Ranh giới quyết định có thể bị dịch chuyển hoặc biến dạng mạnh để bao phủ các điểm nằm xa, ngay cả khi chúng chỉ là nhiễu. Ví dụ, trong bài toán phân loại tuyến tính, một vài điểm dị biệt có thể làm thay đổi đáng kể hướng của siêu phẳng quyết định[9, pp. 185-186].

- **Mất khả năng khái quát hóa.** Khi ranh giới bị kéo lệch theo các điểm dị biệt, mô hình không còn phản ánh đúng quy luật của phần lớn dữ liệu. Điều này dẫn đến sai số huấn luyện thấp nhưng sai số kiểm tra cao[11, p. 112].

Điểm khác biệt quan trọng của outliers so với nhiễu thông thường là ảnh hưởng không cân xứng của chúng lên hàm mất mát, đặc biệt trong các phương pháp sử dụng bình phương sai số. Điều này khiến chỉ một số ít quan sát cũng có thể làm thay đổi đáng kể tham số mô hình.

5.3.2 Sai lầm trong quy trình kiểm chứng

Overfitting không chỉ xuất phát từ độ phức tạp của mô hình mà còn thường xuyên bị “che giấu” hoặc trở nên trầm trọng hơn do các sai sót trong quy trình thực nghiệm, đặc biệt là hiện tượng rò rỉ dữ liệu (data leakage), khi thông tin từ tập kiểm tra vô tình được sử dụng trong quá trình xây dựng mô hình.

Rò rỉ dữ liệu do gộp dữ liệu (Data Leakage from Data Pooling) Việc không phân tách nghiêm ngặt giữa dữ liệu huấn luyện và dữ liệu kiểm tra trong các bước tiền xử lý là nguyên nhân phổ biến dẫn đến đánh giá sai lệch hiệu suất:

- **Chọn đặc trưng (Feature Selection):** Nếu việc chọn đặc trưng được thực hiện trên toàn bộ dữ liệu trước khi chia tập, mô hình sẽ “nhìn thấy trước” các mẫu kiểm tra. Thực nghiệm cho thấy sai lầm này có thể gây sai lệch kết quả lớn hơn so với tính toán[11, pp. 245–247].

Tinh chỉnh tham số không hợp lệ (Improper Parameter Tuning) Một dạng rò rỉ dữ liệu khác xảy ra khi tập kiểm tra được sử dụng để lựa chọn hoặc tối ưu các siêu tham số của mô hình. Trong trường hợp này, tập kiểm tra không còn đóng vai trò đánh giá khách quan, dẫn đến kết quả báo cáo bị lạc quan quá mức và không phản ánh đúng khả năng tổng quát hóa [9, p. 32].

5.3.3 Thủ tục so sánh đa phương (Multiple Comparisons Procedures - MCP)

Trong các thuật toán cảm sinh (induction) và trí tuệ nhân tạo, thủ tục so sánh đa phương được xác định là một trong những nguyên nhân chính dẫn đến hiện tượng overfitting.

Cơ chế gây lỗi Quy trình này thường liên quan đến việc đánh giá đồng thời một số lượng lớn các cấu hình, giả thuyết hoặc tổ hợp thuộc tính dựa trên điểm số từ một hàm mục tiêu và chỉ chọn ra mục có điểm số cao nhất. Khi không gian tìm kiếm quá rộng (tồn tại nhiều giả thuyết), xác suất để một thuộc tính ngẫu nhiên đạt được điểm số cao thuần túy do may mắn sẽ tăng lên đáng kể [1].

Phần 6

Các phương pháp khắc phục Overfitting

6.1 Regularization (Chỉnh hóa/Điều tiết)

Chúng ta định nghĩa **regularization (chuẩn hoá)** là “bất kỳ sự điều chỉnh nào mà ta thực hiện đối với một thuật toán học nhằm giảm lỗi tổng quát hóa (generalization error) nhưng không làm giảm lỗi trên tập huấn luyện (training error) [15, p. 231].”

Có rất nhiều chiến lược regularization. Một số phương pháp đặt thêm các ràng buộc lên mô hình học máy, chẳng hạn như áp đặt giới hạn lên giá trị của các tham số. Một số phương pháp khác thêm các hạng tử vào hàm mục tiêu, có thể được xem như những ràng buộc “mềm” đối với các tham số. Nếu được lựa chọn cẩn thận, các ràng buộc và hình phạt này có thể giúp cải thiện hiệu năng trên tập kiểm tra (test set).

Đôi khi, các ràng buộc và hình phạt này được thiết kế để đưa vào những dạng tri thức tiên nghiệm (prior knowledge) cụ thể. Trong những trường hợp khác, chúng được dùng để thể hiện một xu hướng chung là ưu tiên các mô hình đơn giản hơn nhằm tăng khả năng tổng quát hóa.

Trong một số tình huống, các hình phạt và ràng buộc là cần thiết để biến một bài toán chưa xác định (underdetermined) thành bài toán xác định. Ngoài ra, một số dạng regularization khác, được gọi là **phương pháp tổ hợp (ensemble methods)**, kết hợp nhiều giả thuyết khác nhau cùng giải thích dữ liệu huấn luyện [9, p. 30].

Một kỹ thuật thường được dùng để kiểm soát hiện tượng overfitting như vậy là thêm một hạng tử phạt vào hàm lỗi nhằm ngăn các hệ số đạt giá trị quá lớn.

Hàm lỗi thường dùng:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

6.1.1 L2 Regularization (Weight Decay/Ridge Regression)

Thêm tổng bình phương các trọng số vào hàm lỗi, ép các trọng số tiến về 0 nhưng không bằng 0. Phạt chuẩn tham số L^2 thường được gọi là **suy giảm trọng số (weight decay)** là một trong những dạng phạt chuẩn tham số đơn giản và phổ biến nhất. Chiến lược chính quy hóa này đưa các trọng số tiến gần hơn về gốc tọa độ bằng cách thêm một số hạng chính quy hóa $\Omega(\theta) = \|\mathbf{w}\|^2$ vào hàm mục tiêu. Trong các cộng đồng học thuật khác, chính quy hóa L^2 còn được biết đến với tên gọi **hồi quy ridge (ridge regression)** hoặc **chính quy hóa Tikhonov (Tikhonov regularization)** [15, p. 233][9].

Hàm lỗi được sửa đổi:

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

trong đó:

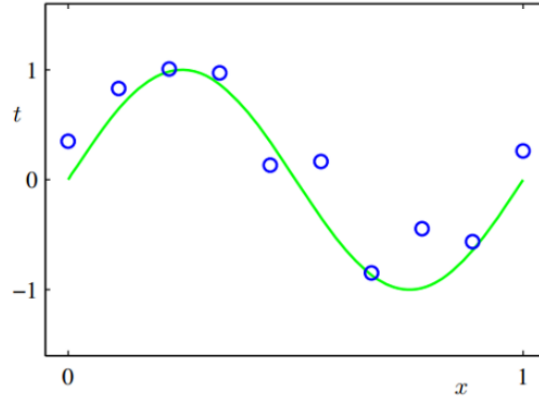
$$\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w} = w_0^2 + w_1^2 + \dots + w_M^2$$

và λ là hệ số điều chỉnh tầm quan trọng giữa số hạng lỗi và số hạng chính quy hóa.

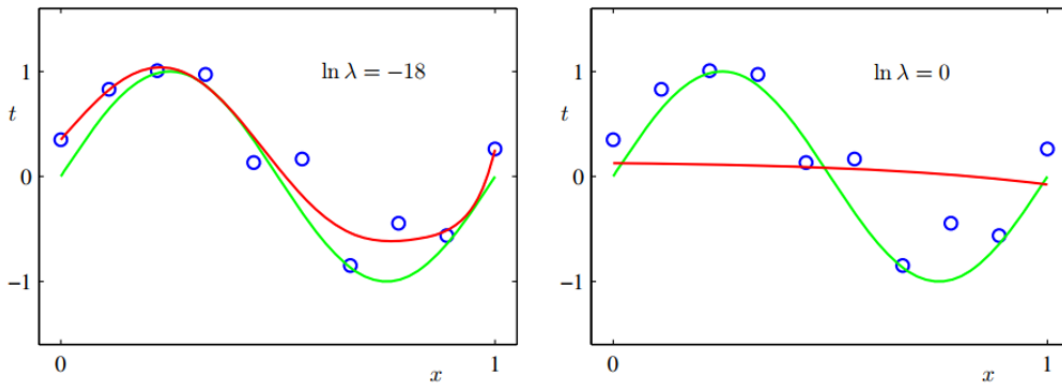
Lưu ý: Thông thường, hệ số w_0 (bias) được bỏ qua khỏi bộ chính quy hóa vì nếu đưa vào, kết quả sẽ phụ thuộc vào cách chọn gốc tọa độ cho biến mục tiêu. Tuy nhiên, cũng có thể đưa w_0 vào nhưng với hệ số chính

quy hóa riêng.

Ví dụ: Đồ thị của một tập dữ liệu huấn luyện gồm $N = 10$ điểm, được biểu diễn bằng các hình tròn màu xanh lam, mỗi điểm bao gồm một quan sát của biến đầu vào x cùng với biến mục tiêu t tương ứng. Đường cong màu xanh lá thể hiện hàm $\sin(2\pi x)$ được sử dụng để sinh ra dữ liệu. Mục tiêu của chúng ta là dự đoán giá trị của t cho một giá trị x mới mà không biết trước đường cong màu xanh lá.



Hình 6.1: Dữ liệu huấn luyện và hàm sin gốc



Hình 6.2: Kết quả khớp đa thức bậc 9 với L2 regularization

Hình cho thấy kết quả khớp đa thức bậc $M = 9$ sử dụng hàm lỗi chính quy hóa L^2 . Với giá trị $\ln \lambda = -18$, hiện tượng over-fitting đã được kiểm soát và thu được biểu diễn gần với hàm $\sin(2\pi x)$ hơn nhiều. Tuy nhiên, nếu λ quá lớn (ví dụ $\ln \lambda = 0$) thì kết quả khớp kém.

6.1.2 L1 Regularization (Lasso)

Thêm tổng giá trị tuyệt đối của các trọng số. Phương pháp này có thể triệt tiêu một số trọng số về đúng bằng 0, giúp lựa chọn đặc trưng (feature selection). Về mặt hình thức, chính quy hóa L^1 trên tham số mô hình $\mathbf{w}$ được định nghĩa là:

$$\Omega(\theta) = \|\mathbf{w}\|_1 = \sum_i |w_i|$$

Khi đó hàm lỗi sửa đổi:

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \alpha \sum_i |w_i|$$

với $\alpha > 0$ là siêu tham số điều khiển cường độ chính quy hóa [15, p. 236][9, p. 165].

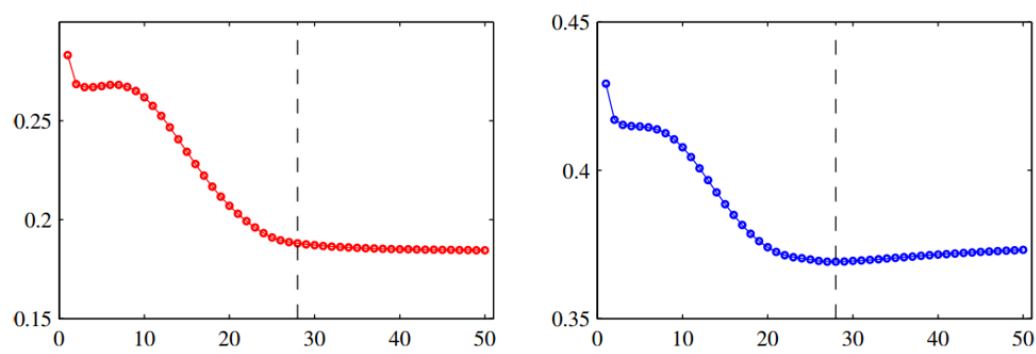
Điểm khác biệt quan trọng: Khác với L^2 có gradient phụ thuộc tuyến tính vào w_i , L^1 đóng góp một hằng số (có dấu) vào gradient, không tỷ lệ với độ lớn của w_i . Điều này dẫn đến các nghiệm không có dạng đại số đẹp như trong L^2 khi xấp xỉ bậc hai.

Ứng dụng: Lựa chọn đặc trưng (Feature Selection) Tính thưa của L^1 (nhiều tham số có giá trị tối ưu bằng 0) được sử dụng rộng rãi như một cơ chế **lựa chọn đặc trưng**. Các trọng số bằng 0 cho biết những đặc trưng tương ứng có thể loại bỏ an toàn, giúp đơn giản hóa bài toán. Mô hình nổi tiếng nhất là **LASSO** (Least Absolute Shrinkage and Selection Operator) – kết hợp phạt L^1 với hồi quy tuyến tính và hàm chi phí bình phương tối thiểu.

6.2 Early Stopping (Dừng sớm)

Một phương pháp hiệu quả nhằm kiểm soát độ phức tạp của mô hình mà không cần sử dụng các kỹ thuật chính quy hóa tường minh là **dừng sớm (early stopping)**. Trong quá trình huấn luyện các mô hình phi tuyến, thuật toán tối ưu thường được thực hiện lặp lại nhằm giảm dần hàm lỗi trên tập huấn luyện. Đối với nhiều phương pháp tối ưu, chẳng hạn như gradient liên hợp, sai số trên tập huấn luyện có xu hướng giảm đơn điệu theo số lần lặp.

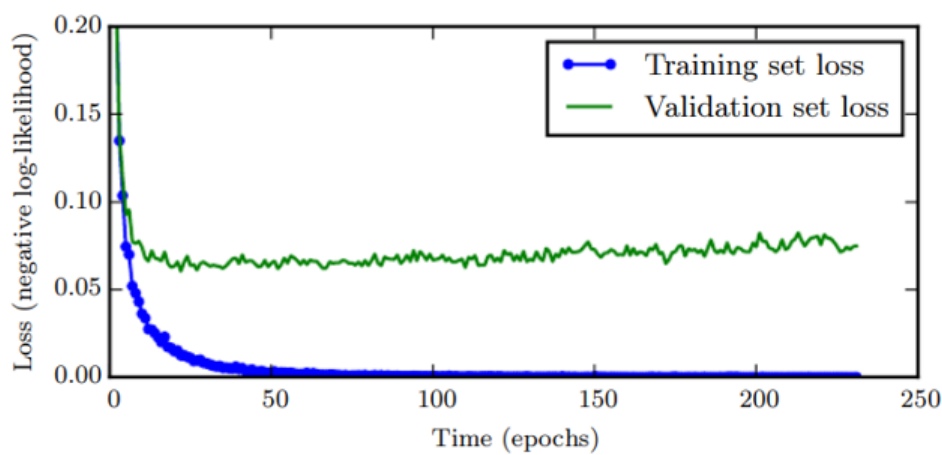
Tuy nhiên, sai số trên một tập dữ liệu độc lập (gọi là tập kiểm chứng – validation set) thường chỉ giảm trong giai đoạn đầu, sau đó bắt đầu tăng lên khi mô hình rơi vào hiện tượng overfitting. Do đó, quá trình huấn luyện nên được dừng lại tại thời điểm mà sai số trên tập kiểm chứng đạt giá trị nhỏ nhất [1][9, p. 279].



Hình 6.3: Sai số huấn luyện và kiểm định theo số bước lặp (epoch)

Hình trên minh họa hành vi của lỗi tập huấn luyện (bên trái) và lỗi tập kiểm định (bên phải) trong một phiên huấn luyện điển hình, như một hàm của bước lặp, đối với tập dữ liệu hình sin. Mục tiêu đạt được hiệu suất tổng quát hóa tốt nhất gợi ý rằng việc huấn luyện nên được dừng tại điểm được chỉ ra bởi các đường đứt nét thẳng đứng, tương ứng với giá trị tối thiểu của lỗi tập kiểm định.

Đồ thị cho thấy các đường cong học tập thể hiện cách độ lỗi log-âm khả năng (negative log-likelihood) thay đổi theo thời gian (được chỉ ra bằng số lần lặp huấn luyện trên tập dữ liệu, hay epoch). Trong ví dụ này, chúng ta huấn luyện một mạng maxout trên MNIST. Quan sát thấy rằng hàm mục tiêu trên tập huấn luyện giảm đều đặn theo thời gian, nhưng độ lỗi trung bình trên tập kiểm định (validation set) cuối cùng bắt đầu tăng trở lại, tạo thành một đường cong hình chữ U bất đối xứng [1][15, p. 240].



Hình 6.4: Đường cong học tập trên MNIST: lỗi log-âm khả năng

6.3 Mở rộng tập dữ liệu (Data Augmentation)

Trong học máy, hiệu năng của mô hình không chỉ phụ thuộc vào thuật toán mà còn chịu ảnh hưởng lớn từ chất lượng và số lượng dữ liệu huấn luyện, đặc biệt trong các bài toán học có giám sát. Quá trình huấn luyện thực chất là việc điều chỉnh các tham số của mô hình nhằm đạt được sự cân bằng giữa độ chính xác trên tập huấn luyện và khả năng tổng quát hóa, từ đó hạn chế hiện tượng overfitting cũng như underfitting.

Để thực hiện việc điều chỉnh này một cách hiệu quả, mô hình cần một lượng dữ liệu đủ lớn. Thông thường, số lượng mẫu cần thiết có xu hướng tăng theo độ phức tạp của mô hình (tức là số lượng tham số cần học). Do đó, việc mở rộng tập dữ liệu có thể cải thiện đáng kể độ chính xác dự đoán, đặc biệt đối với các mô hình phức tạp.

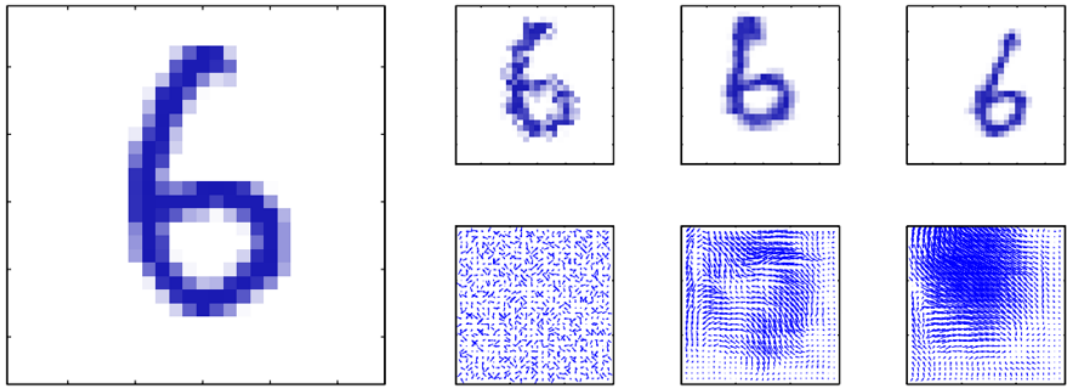
Một kỹ thuật phổ biến để giải quyết vấn đề thiếu dữ liệu là mở rộng tập dữ liệu (data augmentation), trong đó các mẫu dữ liệu mới được tạo ra từ dữ liệu gốc thông qua các phép biến đổi như xoay, lật, thêm nhiễu, hoặc các phép xử lý khác. Phương pháp này đã được chứng minh là hiệu quả trong nhiều lĩnh vực như nhận dạng mẫu và xử lý ảnh, nhờ khả năng cải thiện năng lực tổng quát hóa của mô hình.

Tuy nhiên, việc gia tăng kích thước dữ liệu cũng làm tăng thời gian huấn luyện. Bên cạnh đó, việc thu thập và gán nhãn dữ liệu mới thường tốn kém và đòi hỏi nhiều công sức. Vì vậy, thay vì thu thập dữ liệu hoàn toàn mới, người ta thường tận dụng dữ liệu hiện có để tạo ra các mẫu mới [1][15].

Có bốn hướng tiếp cận chính:

- Thu thập thêm dữ liệu huấn luyện mới
- Thêm nhiễu ngẫu nhiên vào dữ liệu hiện có
- Tái tạo dữ liệu từ tập hiện có thông qua các phép biến đổi
- Sinh dữ liệu mới dựa trên phân phối của tập dữ liệu ban đầu

Trong nhận dạng chữ viết tay, việc dịch chuyển nhẹ các hình ảnh chữ số (translation invariance) giúp mô hình tổng quát hóa tốt hơn. Hình ảnh gốc được hiển thị bên trái. Ở bên phải, hàng trên cùng cho thấy ba ví dụ về chữ số bị làm cong, với các trường dịch chuyển tương ứng được hiển thị ở hàng dưới cùng. Các trường dịch chuyển này được tạo ra bằng cách lấy mẫu các độ dịch chuyển ngẫu nhiên $\Delta x, \Delta y \in (0, 1)$ tại mỗi pixel và sau đó làm mịn bằng tích chập với các Gauss có độ rộng lần lượt là 0.01, 30 và 60 [9, p. 281][15].



Hình 6.5: Minh họa làm cong tổng hợp (synthetic warping) chữ số viết tay

6.4 Validation và Cross-Validation (Kiểm chứng chéo)

Để đánh giá và lựa chọn mô hình một cách khách quan, người ta thường sử dụng tập kiểm định (validation set) nhằm ước lượng sai số trên dữ liệu chưa từng thấy trong quá trình huấn luyện. Về bản chất, tập kiểm định được tạo ra bằng cách tách một phần dữ liệu ra khỏi tập ban đầu và không sử dụng phần này trong quá trình huấn luyện mô hình. Nhờ đó, sai số đo được trên tập kiểm định có thể xem như một xấp xỉ của sai số ngoài mẫu (out-of-sample error).

Mặc dù có vai trò tương tự tập kiểm tra (test set), tập kiểm định vẫn có một điểm khác biệt quan trọng. Cụ thể, tập kiểm định không tham gia trực tiếp vào quá trình huấn luyện, nhưng lại được sử dụng để đưa ra các quyết định trong quá trình học, chẳng hạn như lựa chọn siêu tham số hoặc mô hình tối ưu. Do đó, một khi dữ liệu đã ảnh hưởng đến quá trình học, nó không còn được xem là tập kiểm tra đúng nghĩa. Tuy nhiên, mức độ ảnh hưởng này thường nhỏ, nên tập kiểm định vẫn cung cấp một ước lượng khá tin cậy cho sai số tổng quát hóa.

Về cách thực hiện, giả sử tập dữ liệu ban đầu D được chia thành hai phần: tập huấn luyện D_{train} gồm $N - K$ mẫu và tập kiểm định D_{val} gồm K mẫu. Việc phân chia này có thể được thực hiện bằng nhiều cách khác nhau, miễn là không phụ thuộc vào giá trị của dữ liệu; phổ biến nhất là chọn ngẫu nhiên $N - K$ mẫu cho huấn luyện và sử dụng phần còn lại cho kiểm định.

Bên cạnh đó, kiểm chứng chéo (cross-validation) là một mở rộng của phương pháp trên, trong đó dữ liệu được chia thành nhiều phần và luân phiên sử dụng các phần này cho huấn luyện và kiểm định. Cách tiếp cận này giúp tận dụng tối đa dữ liệu và cung cấp ước lượng ổn định hơn về hiệu năng của mô hình [16, p. 138].

6.4.1 K-fold Cross-Validation

Trong nhiều bài toán thực tế, dữ liệu thường bị hạn chế, do đó cần tận dụng tối đa dữ liệu sẵn có cho quá trình huấn luyện. Tuy nhiên, nếu tập kiểm định quá nhỏ, ước lượng hiệu năng mô hình sẽ trở nên kém ổn định và dễ bị nhiễu. Để giải quyết vấn đề này, phương pháp **K-fold Cross-Validation** được sử dụng.

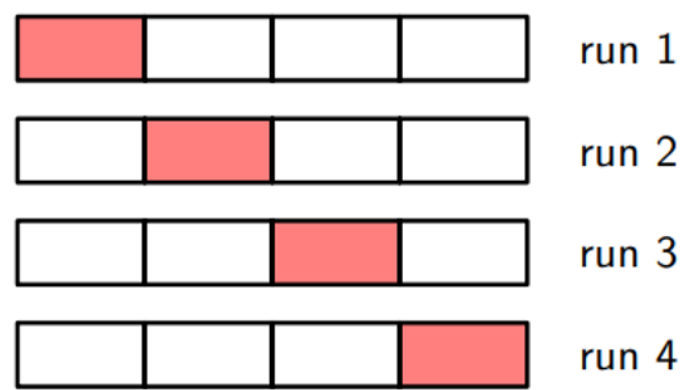
Cụ thể, tập dữ liệu được chia thành S phần (fold) có kích thước xấp xỉ bằng nhau. Ở mỗi lần lặp, một phần được dùng làm tập kiểm định, trong khi $S - 1$ phần còn lại được dùng để huấn luyện. Quá trình này được lặp lại S lần sao cho mỗi phần dữ liệu đều được sử dụng một lần làm tập kiểm định. Nhờ đó, phương pháp này cho phép sử dụng tới $\frac{S-1}{S}$ dữ liệu cho huấn luyện trong mỗi lần lặp, đồng thời tận dụng toàn bộ dữ liệu để đánh giá hiệu năng mô hình.

Trong trường hợp dữ liệu đặc biệt khan hiếm, có thể sử dụng biến thể **Leave-One-Out Cross-Validation**

(**LOOCV**), tương ứng với $S = N$ (với N là số lượng mẫu). Khi đó, mỗi lần lặp chỉ giữ lại đúng một mẫu để kiểm định và sử dụng toàn bộ phần còn lại để huấn luyện.

Mặc dù mang lại ước lượng đáng tin cậy hơn, cross-validation cũng tồn tại một số hạn chế. Trước hết, chi phí tính toán tăng lên đáng kể do mô hình phải được huấn luyện lại S lần. Điều này đặc biệt tốn kém đối với các mô hình có thời gian huấn luyện lớn. Ngoài ra, trong trường hợp mô hình có nhiều siêu tham số (ví dụ nhiều hệ số regularization), việc thử tất cả các tổ hợp tham số có thể dẫn đến số lần huấn luyện tăng theo cấp số nhân, gây khó khăn về mặt tính toán.

Do đó, một hướng tiếp cận lý tưởng là xây dựng các tiêu chí đánh giá chỉ dựa trên dữ liệu huấn luyện, cho phép so sánh nhiều mô hình và siêu tham số trong một lần huấn luyện, đồng thời hạn chế sai lệch do overfitting [9, p. 53][11].



Hình 6.6: Kỹ thuật kiểm định chéo S-fold ($S=4$)

Kỹ thuật kiểm định chéo S-lần (S-fold cross-validation), được minh họa ở đây cho trường hợp $S=4$, bao gồm việc lấy dữ liệu có sẵn và phân chia nó thành S nhóm (trong trường hợp đơn giản nhất, các nhóm có kích thước bằng nhau). Sau đó, $S-1$ nhóm được sử dụng để huấn luyện một tập hợp các mô hình, và các mô hình này được đánh giá trên nhóm còn lại. Quy trình này được lặp lại cho tất cả S lựa chọn khả dĩ cho nhóm bị giữ lại, được chỉ ra ở đây bằng các khối màu đỏ, và các điểm hiệu suất từ S lần chạy sau đó được lấy trung bình.

6.5 Lấy thêm dữ liệu (Data Collection)

Một trong những cách hiệu quả nhất để giảm hiện tượng overfitting là tăng kích thước tập dữ liệu huấn luyện. Tuy nhiên, việc quyết định có nên thu thập thêm dữ liệu hay không cần dựa trên việc đánh giá hiệu năng của mô hình trên cả tập huấn luyện và tập kiểm tra.

Trước hết, cần xem xét hiệu năng trên tập huấn luyện. Nếu mô hình hoạt động kém trên tập này, điều đó cho thấy thuật toán chưa khai thác tốt dữ liệu hiện có. Trong trường hợp này, việc thu thập thêm dữ liệu thường không mang lại hiệu quả. Thay vào đó, nên cải thiện mô hình bằng cách tăng độ phức tạp (ví dụ: thêm lớp hoặc số lượng neuron), hoặc tinh chỉnh các siêu tham số như tốc độ học. Nếu các mô hình phức tạp hơn và thuật toán tối ưu tốt hơn vẫn không cải thiện kết quả, thì nguyên nhân có thể nằm ở chất lượng dữ liệu, chẳng hạn dữ liệu nhiễu hoặc thiếu các đặc trưng quan trọng. Khi đó, cần xem xét thu thập lại dữ liệu sạch hơn hoặc bổ sung thêm đặc trưng phù hợp.

Ngược lại, nếu mô hình đạt hiệu năng tốt trên tập huấn luyện, bước tiếp theo là đánh giá trên tập kiểm tra. Nếu kết quả trên tập kiểm tra cũng tốt, mô hình đã đạt yêu cầu và không cần thay đổi thêm. Tuy nhiên, nếu

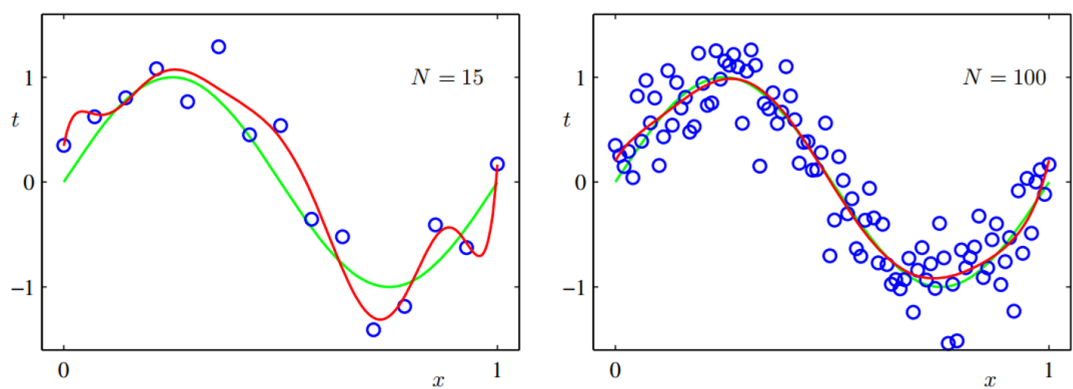
hiệu năng trên tập kiểm tra kém hơn đáng kể so với tập huấn luyện (dấu hiệu của overfitting), thì việc thu thập thêm dữ liệu là một trong những giải pháp hiệu quả nhất.

Khi cân nhắc mở rộng dữ liệu, cần xem xét các yếu tố như chi phí, tính khả thi, cũng như hiệu quả so với các phương pháp khác (ví dụ: regularization hoặc giảm độ phức tạp mô hình). Trong một số lĩnh vực như các hệ thống internet quy mô lớn, việc thu thập dữ liệu thường khả thi và mang lại hiệu quả cao. Ngược lại, trong các lĩnh vực như y tế, việc thu thập dữ liệu có thể tốn kém hoặc khó thực hiện.

Nếu không thể thu thập thêm dữ liệu, có thể áp dụng các giải pháp thay thế như giảm độ phức tạp mô hình hoặc tăng cường regularization (ví dụ: weight decay, dropout). Tuy nhiên, nếu sau khi tinh chỉnh các kỹ thuật này mà khoảng cách giữa lỗi huấn luyện và lỗi kiểm tra vẫn lớn, thì việc bổ sung dữ liệu vẫn là hướng đi cần thiết.

Ngoài ra, khi quyết định thu thập thêm dữ liệu, cần xác định số lượng dữ liệu cần thiết. Một cách tiếp cận hữu ích là xây dựng đường cong học (learning curve) thể hiện mối quan hệ giữa kích thước tập huấn luyện và sai số tổng quát hóa. Dựa trên đó, có thể ước lượng lượng dữ liệu bổ sung cần thiết để đạt được mức hiệu năng mong muốn. Thực tế cho thấy, việc chỉ tăng một lượng nhỏ dữ liệu thường không mang lại cải thiện đáng kể; do đó, nên thử nghiệm với các mức dữ liệu tăng theo cấp số nhân (ví dụ: gấp đôi sau mỗi lần thử nghiệm).

Trong trường hợp không thể mở rộng dữ liệu, việc cải thiện hiệu năng mô hình sẽ phụ thuộc vào việc phát triển các thuật toán học tốt hơn, đây thường là hướng nghiên cứu chuyên sâu hơn là áp dụng thực tiễn [9][15].



Hình 6.7: Ảnh hưởng của kích thước tập dữ liệu lên overfitting (đa thức bậc 9)

Đồ thị của các nghiệm thu được bằng cách tối thiểu hóa hàm lỗi tổng bình phương sử dụng đa thức bậc $M=9$ cho $N=15$ điểm dữ liệu (đồ thị bên trái) và $N=100$ điểm dữ liệu (đồ thị bên phải). Chúng ta thấy rằng việc tăng kích thước tập dữ liệu sẽ làm giảm vấn đề quá khớp (over-fitting).

Tài liệu tham khảo

- [1] Xue Ying. “An Overview of Overfitting and its Solutions”. *Journal of Physics: Conference Series* 1168 (2019), 022022. DOI: [10.1088/1742-6596/1168/2/022022](https://doi.org/10.1088/1742-6596/1168/2/022022).
- [2] Thomas G. Dietterich. “Overfitting and Undercomputing in Machine Learning”. *ACM Computing Surveys* 27.3 (1995), 326–327. DOI: [10.1145/212094.212114](https://doi.org/10.1145/212094.212114).
- [3] Yaser S. Abu-Mostafa. *Learning From Data – Lecture 11: Overfitting*. Accessed: 2026-04-24. 2012. URL: <https://work.caltech.edu/lectures.html#lectures>.
- [4] Konstantinos Poulinakis **and others**. “Machine-Learning Methods on Noisy and Sparse Data”. *Mathematics* 11.1 (2023), 236. DOI: [10.3390/math11010236](https://doi.org/10.3390/math11010236).
- [5] Stuart Geman, Elie Bienenstock **and** René Doursat. “Neural Networks and the Bias/Variance Dilemma”. *Neural Computation* 4 (january 1992), 1–58. DOI: [10.1162/neco.1992.4.1.1](https://doi.org/10.1162/neco.1992.4.1.1).
- [6] Pedro Domingos. “A Unifeid Bias-Variance Decomposition and its Applications.”. **january** 2000, 231–238.
- [7] Nitish Shirish Keskar **and others**. *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*. 2017. arXiv: [1609.04836 \[cs.LG\]](https://arxiv.org/abs/1609.04836). URL: <https://arxiv.org/abs/1609.04836>.
- [8] Jingfeng Wu **and others**. “On the Noisy Gradient Descent that Generalizes as SGD”. *Proceedings of the 37th International Conference on Machine Learning*. **by editor** Hal Daumé III **and** Aarti Singh. 119. *Proceedings of Machine Learning Research*. PMLR, 13–18 Jul 2020, 10367–10376. URL: <https://proceedings.mlr.press/v119/wu20c.html>.
- [9] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [10] Andrius Vabalas **and others**. “Machine learning algorithm validation with a limited sample size”. *PLoS ONE* 14.11 (2019), e0224365. DOI: [10.1371/journal.pone.0224365](https://doi.org/10.1371/journal.pone.0224365).
- [11] Trevor Hastie, Robert Tibshirani **and** Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2 **edition**. Springer, 2009.
- [12] Gilad Dar, Vidya Muthukumar **and** Richard G. Baraniuk. “Interpolating Classifiers Make Few Mistakes”. *arXiv preprint arXiv:2109.02355* (2021). DOI: [10.48550/arXiv.2109.02355](https://doi.org/10.48550/arXiv.2109.02355).
- [13] Yehuda Dar, Vidya Muthukumar **and** Richard G Baraniuk. “A Farewell to the Bias-Variance Tradeoff? An Overview of the Theory of Overparameterized Machine Learning”. *arXiv preprint arXiv:2109.02355* (2021). DOI: [10.48550/arXiv.2109.02355](https://doi.org/10.48550/arXiv.2109.02355).
- [14] Mikhail Belkin **and others**. “Reconciling modern machine-learning practice and the classical bias-variance trade-off”. *Proceedings of the National Academy of Sciences* (2019). DOI: [10.1073/pnas.1903070116](https://doi.org/10.1073/pnas.1903070116).
- [15] Ian Goodfellow, Yoshua Bengio **and** Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [16] Yaser S. Abu-Mostafa, Malik Magdon-Ismail **and** Hsuan-Tien Lin. *Learning From Data: A Short Course*. AMLBook.com, 2012.